

CHIANG MAI UNIVERSITY
Master of Science (Software Engineering)
College of Arts, Media and Technology
2nd Semester / Academic Year 2025
SE713 Backend Development

ORM

วัตถุประสงค์ ทดลองสร้าง ORM application

หากแลกเปลี่ยนหน้ายังไม่เสร็จสามารถ download file เพื่อทำ lab ต่อได้[ที่นี่](#)

Task 1 การติดตั้ง Prisma และสร้าง Table อัตโนมัติ

1. เพื่อการติดตั้ง Prisma ต้องติดตั้งผ่าน npm โดยใช้คำสั่ง

```
npm install prisma tsx --save-dev
npm install @prisma/client @prisma/adapters-pg-dotenv
```

2. เริ่มให้ project ใช้ Prisma โดยใช้คำสั่ง

```
npx prisma init
```

จะมีไฟล์ `prisma/schema.prisma` สร้างขึ้นมาโดยอัตโนมัติ และ ไฟล์ `.env` จะถูกเพิ่มข้อความลงไปโดยอัตโนมัติ

3. แก้ไข `tsconfig.json` เพื่อให้ทำงานกับ prisma ได้ โดยแก้ไขเป็นดังนี้

```
{
  "compilerOptions": {
    // File Layout
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "ESNext",
    "moduleResolution": "node",
    "target": "ES2023",
    "strict": true,
    "esModuleInterop": true,
    "ignoreDeprecations": "6.0"
  }
}
```

4. แก้ไข `package.json` โดยเพิ่ม "type": "module" ใน `package.json` การจัดการนี้

```
"name": "713-2015-backend",
"version": "1.0.0",
"main": "index.js",
+ "type": "module",
```

```
"scripts": {
```

5. เชื่อมต่อกับฐานข้อมูล mysql ในเครื่อง โดยตรวจสอบไฟล์

prisma/schema.prisma ว่ามี content ดังนี้

```
datasource db {  
  provider = "postgresql"  
}
```

จากนั้นแก้ไข DATABASE_URL ใน .env เพื่อระบุตำแหน่งของฐานข้อมูลที่ต้องการติดต่อ

```
DATABASE_URL="postgres://admin:admin123@localhost:5432/event?schema=public"
```

6. เพิ่ม model ของฐานข้อมูลที่ต้องการใน prisma/schema.prisma ดังนี้

```
model event {  
  id          Int          @id @default(autoincrement())  
  category    String  
  title       String  
  description  String  
  location    String  
  date        String  
  time        String  
  petsAllowed Boolean  
  organizer   String  
}
```

และแก้ไข ตัว database เป็น

```
datasource db {  
  provider = "postgresql"  
}
```

7. เชื่อมต่อกับฐานข้อมูลโดยใช้คำสั่ง

```
npx prisma migrate dev --name init
```

เมื่อทำสำเร็จแล้วให้เปิดดูฐานข้อมูลจะพบ table event อยู่ที่มีโครงสร้างที่สร้างมาโดยอัตโนมัติ

Task 2 การ seed data เข้าสู่ฐานข้อมูล

1. ลง component เพื่อใช้ในการเชื่อมต่อฐานข้อมูล โดยใช้คำสั่ง

```
npm install @prisma/client
```

```
npm install @prisma/extension-accelerate
```

2. สร้าง Client โดยใช้คำสั่ง `npx prisma generate` เพื่อสร้าง db client ใหม่

3. สร้าง folder `src/lib` แล้วตั้งชื่อว่า `prisma.ts` เพื่อสร้าง prisma library ไว้ใช้

```
+import "dotenv/config";
+import { PrismaPg } from '@prisma/adapter-pg'
+import { PrismaClient } from '../generated/prisma/client'

+const connectionString = `${process.env.DATABASE_URL}`

+const adapter = new PrismaPg({ connectionString })
+const prisma = new PrismaClient({ adapter })

+export { prisma }
```

4. สร้าง ไฟล์ `src/db/createEvents.ts` เพื่อสร้างการ เพิ่มข้อมูลลงฐานข้อมูล

```
+import { prisma } from '../lib/prisma'
+
+export async function createEvents() {
+  const events = [
+    {
+      category: "Music",
+      title: "Concert",
+      description: "A live concert",
+      location: "London",
+      date: "2021-07-01",
+      time: "19:00",
+      petsAllowed: false,
+      organizer: "Live Nation"
+    },
+    {
+      category: "Music",
+      title: "Festival",
+      description: "A music festival",
+      location: "Manchester",
+      date: "2021-07-15",
+      time: "12:00",
+      petsAllowed: true,
+      organizer: "Festival Republic"
+    },
+    {
+      category: "Sports",
+      title: "Football Match",
+      description: "A football match",
+      location: "Liverpool",
+      date: "2021-08-01",
+      time: "15:00",
+      petsAllowed: false,

```

```

+     organizer: "Premier League"
+   },
+   {
+     category: "Music",
+     title: "Jazz Night",
+     description: "An evening of smooth jazz",
+     location: "New Orleans",
+     date: "2021-09-10",
+     time: "19:00",
+     petsAllowed: true,
+     organizer: "Jazz Fest"
+   },
+   {
+     category: "Theatre",
+     title: "Shakespeare in the Park",
+     description: "A performance of Hamlet",
+     location: "Central Park",
+     date: "2021-10-05",
+     time: "18:00",
+     petsAllowed: false,
+     organizer: "NYC Theatre Group"
+   },
+   {
+     category: "Food",
+     title: "Food Truck Festival",
+     description: "A variety of food trucks offering delicious meals",
+     location: "San Francisco",
+     date: "2021-11-20",
+     time: "12:00",
+     petsAllowed: true,
+     organizer: "Foodie Events"
+   }
+ ];
+
+ for (const event of events) {
+   await prisma.event.create({
+     data: {
+       category: event.category,
+       title: event.title,
+       description: event.description,
+       location: event.location,
+       date: event.date,
+       time: event.time,
+       petsAllowed: event.petsAllowed,
+       organizer: event.organizer
+     }
+   });
+ }
+
+ console.log("Database has been initialized with events.");
+}

```

สร้าง seed.ts บน prisma folder เพื่อใช้ในการ seed ข้อมูล โดยเลือกใช้ functions ที่สร้างจาก db/createEvents.ts

```
+import {createEvents} from '../src/db/createEvents';
+import {prisma} from '../src/lib/prisma'
+createEvents()
+  .catch((e) => {
+    console.error(e);
+    process.exit(1);
+  })
+  .finally(async () => {
+    await prisma.$disconnect();
+  });
```

5. รันคำสั่ง `npx tsx prisma/seed.ts` เพื่อเรียกให้คำสั่ง seed ที่ระบุไว้ทำงาน
สังเกตว่าตอนนี้ในฐานข้อมูลควรมีข้อมูลเพิ่มเข้าไปในฐานข้อมูลแล้ว
6. เพื่อให้การเรียกใช้คำสั่ง seed ง่ายสามารถเพิ่ม script เข้าไปใน

prisma.config.ts ดังนี้

```
migrations: {
  path: "prisma/migrations",
  seed: 'tsx prisma/seed.ts'
},
```

จากนั้นเรียกใช้คำสั่งโดยใช้ `npx prisma db seed` เพื่อเพิ่มข้อมูลได้

7. ทั้งนี้ข้อมูลอาจมีการซ้ำ เพราะเรามีการ seed data ไปแล้วสองครั้งสามารถแก้ไขได้โดยลบ database เดิมแล้วใช้คำสั่ง `npx prisma migrate dev --name init`

เพื่อสร้างฐานข้อมูลใหม่และ run seed อีกครั้ง
หรือแก้ไข seed.ts ให้ลบข้อมูลในฐานข้อมูลทั้งหมดดังนี้

```
prisma.event.deleteMany().then(() => {
  createEvents()
    .catch((e) => {
      console.error(e);
      process.exit(1);
    })
    .finally(async () => {
      await prisma.$disconnect();
    });
});
```


Task 3 การเรียกใช้งาน functions เบื้องต้น

1. สร้าง repository ใหม่สำหรับการเชื่อมต่อข้อมูลโดยใช้ prisma โดยสร้างไฟล์ repository/eventRepositoryPrisma.ts และเพิ่มข้อมูลดังนี้

```
+import {prisma} from '../lib/prisma';
+import type Event from '../models/Event';
+
+export function getCategory(category: string) {
+  return prisma.event.findMany({
+    where: { category },
+  });
+}
+
+export function getAllEvents() {
+  return prisma.event.findMany();
+}
+
+export function getEventById(id: number) {
+  return prisma.event.findUnique({
+    where: { id },
+  });
+}
+
+export function addEvent(newEvent: Event){
+  return prisma.event.create({
+    data: {
+      category: newEvent.category,
+      title: newEvent.title,
+      description: newEvent.description,
+      location: newEvent.location,
+      date: newEvent.date,
+      time: newEvent.time,
+      petsAllowed: newEvent.petsAllowed,
+      organizer: newEvent.organizer
+    }
+  });
+}
```

2. จากนั้นแก้ไข EventService เพื่อเรียกใช้ function นี้แทนของเดิม

```
import type Event from "../models/Event";
-import * as repo from "../repository/EventRepositoryDb";
+import * as repo from "../repository/eventRepositoryPrisma";
```

3. ขณะเนื่องจาก Type ของ Event ที่ได้ปัจจุบัน เป็น Type ที่ได้จาก prisma model ไม่ใช่ Event ของเรานั้นจึงควรแก้ไข โดยนำ return type ของทุก functions ใน repository และ service ออกจากทุกฟังก์ชัน เพื่อใช้ type ที่ถูกสร้าง (imply) จาก prisma model เท่านั้น ตัวอย่างเช่น เช่นการแก้ไข function getEventById

```
-export function getEventById(id: number): Promise<Event | undefined> {
+export function getEventById(id: number){
+  return repo.getEventById(id);
+}
```

4. แก้ไข Event Type ใน EventRepository และ EventService โดยแก้ไขการ import เป็นดังนี้

```
-import type Event from '../models/Event';  
+import type {eventModel as Event} from "../generated/prisma/models/event";
```

5. ทดลองเรียกใช้ api ควรจะต้องทำงานได้เหมือนเดิม

Task 4 One To Many Relationship

1. จากนั้นทดลองสร้าง OneToMany Relationship โดยให้มีหน่วยงานที่จัดโครงการ (organizer) ให้แต่ละ organizer สามารถสร้าง event ได้หลาย event. แก้ไข model ใน prisma/schema.prisma ดังนี้

```
time          String
petsAllowed Boolean
- organizer   String
+ organizerId Int?
+ organizer   organizer? @relation(fields: [organizerId], references: [id])
+
+
+model organizer{
+  id          Int          @id @default(autoincrement())
+  name        String
+  events      event[]
+}
```

2. ลบฐานข้อมูลเดิม (เนื่องจากการเปลี่ยนชนิดของ organizer) จากนั้นสร้าง table ใหม่ โดยใช้ command

```
npx prisma migrate dev --name addOrganizer
```

จะพบว่า prisma สร้าง table event และ organizer ให้ พร้อมทั้ง มีการกำหนด constraint ให้เรียบร้อย

3. จากนั้นปรับปรุง Prisma Client เพื่อให้สามารถเชื่อมต่อกับ ตาราง organization ด้วย คำสั่ง

```
npx prisma generate
```

4.

แก้ไข db/createEvents เพื่อเพิ่มข้อมูล Organizer ลงในฐานข้อมูล โดยเพิ่มข้อมูลของ Organization ใน list ก่อน และเพิ่มข้อมูลลงในฐานข้อมูล

```
export async function createEvents() {
+  const ChiangMaiOrg = await prisma.organizer.create({
+    data: {
+      name: 'Chiang Mai'
+    }
+  })
+
+  const cmuOrg = await prisma.organizer.create({
+    data: {
+      name: 'Chiang Mai University'
+    }
+  })
+
+  const camtOrg = await prisma.organizer.create({
+    data: {
+      name: 'CAMT'
+    }
+  })

  const events = [
```

ทั้งนี้ แก้ไขค่า organizer จากเดิมเป็น organize object

```
export async function createEvents() {
  const events = [

    {
      category: "Music",
      title: "Concert",
      ...
      location: "London",
      date: "2021-07-01",
      time: "19:00",
      petsAllowed: false,
      - organizer: "Live Nation"
      + organizer: ChiangMaiOrg
    },
    {
      category: "Music",
      ...
      location: "Manchester",
      date: "2021-07-15",
      time: "12:00",
      petsAllowed: true,
      + organizer: cmuOrg
    },
    {
      category: "Sports",
      ...
      location: "Liverpool",
      date: "2021-08-01",
      time: "15:00",
      petsAllowed: false,
      - organizer: "Premier League"
      + organizer: camtOrg
    },
    {
      category: "Music",
      ...
      location: "New Orleans",
      date: "2021-09-10",
      time: "19:00",
      petsAllowed: true,
      - organizer: "Jazz Fest"
      + organizer: ChiangMaiOrg
    },
    {
      category: "Theatre",
      ...
      location: "Central Park",
      date: "2021-10-05",
      time: "18:00",
      petsAllowed: false,
      - organizer: "NYC Theatre Group"
      + organizer: cmuOrg
    },
    {
      category: "Food",
      ...
      location: "San Francisco",
```

```

        date: "2021-11-20",
        time: "12:00",
        petsAllowed: true,
-       organizer: "Foodie Events"
+       organizer: cmuOrg
    }
  ];

-   for (const event of events) {
+   for (const event of events) {
      await prisma.event.create({
+     data: {
+       category: event.category,
+       title: event.title,
+       description: event.description,
+       location: event.location,
+       date: event.date,
+       time: event.time,
+       petsAllowed: event.petsAllowed,
+       organizerId: event.organizer.id
+     }

    });
  });

```

5. แก้ไข seed.ts เพื่อให้ลบ data เดิมในฐานข้อมูล และและ เพิ่มข้อมูลใหม่ โดยครั้งนี้จะทดลองเขียนในรูปแบบ async/await ดังนี้

```

import {createEvents} from '../src/db/createEvents';
import {prisma} from '../src/lib/prisma'

const seedData = async ()=> {
  await prisma.event.deleteMany();
  await prisma.organizer.deleteMany();
  await createEvents();
}
await seedData();

```

6. เนื่องจาก type ของ organizer ใน event เปลี่ยนไป ขอให้แก้ไข addEvents ให้ทำงานได้ก่อนดังนี้

```

export function addEvent(newEvent: Event): Promise<Event> {
  return prisma.event.create({
    data: {
      category: newEvent.category,
      title: newEvent.title,
      description: newEvent.description,
      location: newEvent.location,
      date: newEvent.date,
      time: newEvent.time,
      petsAllowed: newEvent.petsAllowed,
-     organizer: newEvent.organizer
    }
  });
}

```

7. ทดลอง ลบ database สร้าง database ใหม่ แล้วลอง seed data เพื่อดูว่าตอนนี้ข้อมูล event และ organizer มีการเชื่อมต่อกันแล้วสามารถบันทึกลงฐานข้อมูลได้แล้ว

Task 5 One To Many Query

1. เมื่อเราตรวจสอบ EventRepositoryPrisma.ts จะพบว่ามีคำเตือนว่า type ไม่ถูกต้อง เนื่องจาก Type ของ Event ที่ได้ปัจจุบัน เป็น Type ที่ได้จาก prisma model ไม่ใช่ Event model ดังนั้นจึงควรแก้ไข โดยนำ return type ของทุก functions ใน repository และ service ออก เพื่อใช้ type ที่ถูกสร้างจาก prisma model เท่านั้น
2. ทดลองเรียกข้อมูลจาก api localhost:8080/event จะเห็นว่าไม่มีข้อมูล organizer มาแสดงในข้อมูลของ event เราต้องสั่งให้ prisma query ข้อมูลของ organizer มาแสดงด้วย โดยเปิดไฟล์

repository/eventRepositoryPrisma.ts เพิ่ม function นี้

```
+export function getAllEventsWithOrganizer() {  
+  return prisma.event.findMany({  
+    include: { organizer: true },  
+  });  
+}
```

3. แก้ไขส่วน service ของ getAllEvents เพื่อเรียกใช้ function ที่สร้างมาใหม่

```
export function getAllEvents() {  
-  return repo.getAllEvents();  
+  return repo.getAllEventsWithOrganizer();  
}
```

4. ทดสอบ query ข้อมูล ควรจะเห็นข้อมูลของ organizer ใน event
5. แต่เราก็ก็น่าจะเห็น organizerId ที่แยกออกมา หากไม่ต้องการแสดง organizerId ให้เพิ่ม code ดังนี้ใน function getAllEventsWithOrganizer

```
export function getAllEventsWithOrganizer() {  
  return prisma.event.findMany({  
    include: { organizer: true },  
+    omit: { organizerId: true }  
  });  
}
```

6. ทดสอบ query ข้อมูล ไม่ควรเห็น organizerId ใน event
7. ทดลองทำให้ findById สามารถแสดงค่าของ organizer ออกมาด้วย

Task 6 Fields Selective Query

1. เลือกให้การแสดงผล organizer แสดงแค่ชื่อนั้นโดยแก้ไข function getAllEventsWithOrganizer ดังนี้

```
export function getAllEventsWithOrganizer(): Promise<Event[]> {  
  return prisma.event.findMany({  
    - include: { organizer: true },  
    + include: {  
    +   organizer: {  
    +     select: {  
    +       name: true,  
    +     },  
    +   },  
    + },  
    omit: {organizerId: true}  
  });  
}
```

ทดลองเรียก api เพื่อดูผลลัพธ์ที่ได้

2. ทั้งนี้หากต้องการเลือกบาง field ใน event ด้วย โดยไม่ต้องการแสดง organizationId แยกออกมา สามารถแก้ไขจาก include เป็น select ดังนี้

```
export function getAllEventsWithOrganizer() {  
  return prisma.event.findMany({  
    select: {  
      id: true,  
      category: true,  
      organizerId: false,  
      organizer: {  
        select: {  
          name: true  
        }  
      }  
    }  
  });  
}
```

3. ทดลอง ดูค่าที่ได้จาก api

4. ทดลองแก้ไขส่วนของ findEventById ให้แสดง ชื่อ เวลา และ organizer id เท่านั้น

Task 7 Many To Many Relations

1. อีกลักษณะความสัมพันธ์คือ Many To Many Relationship โดยตอนนี้จะสร้างผู้เข้าร่วม (participant) ให้สามารถเข้าร่วมกับ events ก็ได้ และ ให้แต่ละ event บันทึกจำนวนผู้เข้าร่วมได้ โดยปรับส่วนของ

prisma/schema.prisma ได้ดังนี้

```
model event{
  ...
  + organizer      organizer?    @relation(fields: [organizerId], references:
[id])
  + participants participant[]
}
+
+model participant {
+ id      Int      @id @default(autoincrement())
+ name    String
+ email   String
+ events  event[]
+}
```

2. ทำการปรับปรุง ฐานข้อมูล โดยใช้คำสั่ง

`npx prisma migrate dev --name addParticipants`

3. ปรับปรุง Client เพื่อให้สามารถเชื่อมต่อกับตารางได้ด้วยคำสั่ง

`npx prisma generate`

4. สร้างไฟล์ `db/createParticipants.ts` เพื่อเพิ่ม participant และ ผูก event กับ participant เข้าด้วยกัน ดังนี้

```
+import {prisma} from '../lib/prisma';
+
+export async function createParticipants() {
+  const participants = [
+    {
+      name: 'John Doe',
+      email: 'john.doe@example.com',
+    },
+    {
+      name: 'Jane Smith',
+      email: 'jane.smith@example.com',
+    },
+    {
+      name: 'Alice Johnson',
+      email: 'alice.johnson@example.com',
+    },
+    {
+      name: 'Bob Brown',
+      email: 'bob.brown@example.com',
+    },
+    {
+      name: 'Charlie Davis',
+      email: 'charlie.davis@example.com',
+    },
+  ];
+
+  for (const participant of participants) {
+    await prisma.participant.create({
+      data: participant,
```

```

+     });
+   }
+
+   const responseParticipants = await prisma.participant.findMany();
+   const responseEvents = await prisma.event.findMany();
+
+   Promise.all([addEvent(responseParticipants[0].id, responseEvents[0].id),
+     addEvent(responseParticipants[0].id, responseEvents[1].id),
+     addEvent(responseParticipants[0].id, responseEvents[2].id),
+     addEvent(responseParticipants[1].id, responseEvents[3].id),
+     addEvent(responseParticipants[1].id, responseEvents[4].id),
+     addEvent(responseParticipants[1].id, responseEvents[5].id),
+     addEvent(responseParticipants[2].id, responseEvents[0].id),
+     addEvent(responseParticipants[2].id, responseEvents[1].id),
+     addEvent(responseParticipants[3].id, responseEvents[2].id),
+     addEvent(responseParticipants[4].id, responseEvents[3].id),
+     addEvent(responseParticipants[4].id, responseEvents[4].id),
+     addEvent(responseParticipants[1].id,
+       responseEvents[5].id)]).then(() => {
+     Promise.all([
+       prisma.participant.findMany({
+         include: {
+           events: true
+         }
+       }),
+       prisma.event.findMany({
+         include: {
+           participants: true
+         }
+       })
+     ]).then(([participantsWithEvents, eventsWithParticipants]) => {
+       console.log("Participants with their Events:",
+         JSON.stringify(participantsWithEvents, null, 2));
+       console.log("Events with their Participants:",
+         JSON.stringify(eventsWithParticipants, null, 2));
+     })
+   })
+ }
+
+}
+
+async function addEvent(participantId: number, eventId: number) {
+  await prisma.participant.update({
+    where: { id: participantId },
+    data: {
+      events: {
+        connect: {
+          id: eventId,
+        },
+      },
+    },
+  });
+}

```

5. ที่ seed.ts แก้ไขให้ เรียกใช้ function createParticipant เพื่อเพิ่มข้อมูล Participant เท่านั้น

```

import {createEvents} from '../src/db/createEvents';
import {prisma} from '../src/lib/prisma'

```

```

+ import {createParticipants} from "../src/db/createParticipants";

const seedData = async ()=> {
+   await prisma.participant.deleteMany();
  await prisma.event.deleteMany();
  await prisma.organizer.deleteMany();
  await createEvents();
+   await createParticipants()
}

```

6. ทดลอง run seed จะเห็นผลลัพธ์ที่ได้จากการ link ข้อมูลว่าใน event มี participant อยู่

7. ในส่วนของ `getAllEventsWithOrganizer` ใน `eventRepositoryPrisma.ts` เพิ่มการ select participants ลงไป

```

    },
-   },
+   participants: {
+     select: {
+       id: true,
+       name: true,
+       email: true,
+       events: true,
+     },
+   },
+   }
+ });

```

8. ทดลองยิง `api /events` เพื่อดูผลลัพธ์ว่าได้ข้อมูลที่มี participant มาด้วย

Task Homework1 Create a new Prisma Project

1. สร้าง project ใหม่ (Repository ใหม่) เพื่อสร้าง application ห้องสมุด โดยจะมีการเก็บข้อมูลหนังสือ (ชื่อหนังสือ ISBN หนังสือ หมวดหนังสือ) โดยหนังสือแต่ละเล่มจะมีผู้แต่งหนึ่งท่าน แต่ผู้แต่งหนึ่งท่านจะสามารถแต่งหนังสือกี่เล่มก็ได้ ข้อมูลผู้แต่งประกอบด้วย ชื่อ นามสกุล ต้นสังกัด โดยจะมีสมาชิก เป็นผู้ที่สามารถยืมหนังสือได้ โดยสมาชิกจะประกอบไปด้วย รหัสสมาชิก ชื่อ นามสกุล และหมายเลขโทรศัพท์ โดยหนึ่งคนสามารถยืมกี่เล่มก็ได้ และหนังสือต้องเก็บประวัติการยืม โดยประวัติการยืมจะระบุผู้ยืม จำนวนหนังสือที่ถูกยืมไป และ กำหนดเวลาการคืน และเวลาในการคืนจริง ทั้งนี้ กำหนดเวลาการคืนและเวลาที่คืนจริงจะเป็นเวลาสำหรับหนังสือแต่ละเล่มที่คืนแยกกันไป
2. ให้สร้าง endpoint สำหรับการเรียกข้อมูลหนังสือทั้งหมด การค้นหาหนังสือตามรายชื่อ การแสดงรายชื่อผู้แต่งทั้งหมด การแสดงสมาชิกตามชื่อ และตามหมายเลขสมาชิก และการค้นหาหนังสือที่มีกำหนดการคืนในวันที่กำหนด และหนังสือที่ยังไม่ได้คืน
3. ให้มีการเตรียมข้อมูล (seed ข้อมูล) ตามความเหมาะสม

ทำการบ้านแล้วก็เอาไปรวมกับงานของข้อต่อๆ ไปด้วย
งานนี้คิดเป็น ร้อยละ 5 ของคะแนนรวมทั้งหมด

Task 8 การ refactor route

1. เพื่อให้ server.ts เรียบร้อย ลดความซับซ้อนสามารถปรับ route ได้ โดยใช้ router object

สร้าง โฟลเดอร์ routes และสร้าง ไฟล์ EventRoute.ts โดยมี เนื้อหาตามนี้
สามารถถาม copilot ว่า refactor event route to router เพื่อได้ code ตัวอย่าง
ได้

```
+import express, {Request, Response} from "express";
+import * as service from "../services/EventService";
+import type {eventModel as Event} from "../generated/prisma/models/event";
+
+import exp from "constants";
+
+const router = express.Router();
+
+router.get("/events/:id", async (req, res) => {
+  const id = parseInt(req.params.id);
+  const event = await service.getEventById(id);
+  if (event) {
+    res.json(event);
+  } else {
+    res.status(404).send("Event not found");
+  }
+});
+
+router.get("/events", async (req, res) => {
+  if (req.query.category) {
+    const category = req.query.category as string;
+    const filteredEvents = await service.getEventByCategory(category);
+    res.json(filteredEvents);
+  } else {
+    res.json(await service.getAllEvents());
+  }
+});
+
+router.post("/events", async (req, res) => {
+  const newEvent: Event = req.body;
+
+  res.json(await service.addEvent(newEvent));
+});
+
+export default router;
```

2. แก้ไข server.ts เพื่อเพิ่ม route ไปใช้ และลบ route ที่ซ้ำซ้อนเดิมออก

```
import express, { Request, Response } from "express";
-import { getAllEvents, getEventByCategory, getEventById, addEvent } from
"./services/EventService";
-import type { Event } from "../models/Event";
+import eventRoute from './routes/EventRoute';

import { uploadFile } from './services/UploadFileService';
const app = express();
app.use(express.json());
```

```

+app.use('/',eventRoute);
const port = 3000;
...
-app.get("/events", async(req, res) => {
-  if (req.query.category) {
-    const category = req.query.category;
-    const filteredEvents = await getEventByCategory(category as string);
-    res.json(filteredEvents);
-  } else {
-    res.json(await getAllEvents());
-  }
-});
-

-app.get("/events/:id", async (req, res) => {
-  const id = parseInt(req.params.id);
-  const event = await getEventById(id);
-  if (event) {
-    res.json(event);
-  } else {
-    res.status(404).send("Event not found");
-  }
-});
-

-app.post("/events", async (req, res) => {
-  const newEvent: Event = req.body;
-  await addEvent(newEvent);
-  res.json(newEvent);
-});
...

app.listen(port, () => {
  console.log(`App listening at http://localhost:${port}`)
})

```

3. ตอนนี้ server.ts จะดูเรียบร้อยขึ้น แต่ใน EventRoute.ts จะยังมี /event อยู่มากเกินไป ซึ่งสามารถปรับให้สามารถนำไป reuse ได้เพิ่มเติมโดยแก้ไข EventRoute.ts

```

import exp from "constants";
const router = express.Router();

-router.get("/events", async(req, res) => {
+router.get("/", async(req, res) => {
  if (req.query.category) {
    const category = req.query.category;
    const filteredEvents = await getEventByCategory(category as string);
...
-router.get("/events/:id", async (req, res) => {
+router.get("/:id", async (req, res) => {
  const id = parseInt(req.params.id);
  const event = await getEventById(id);
  if (event) {
...

-router.post("/events", async (req, res) => {
+router.post("/", async (req, res) => {

```

```
const newEvent: Event = req.body;
await addEvent(newEvent);
```

แก้ไข server.ts

```
const app = express();
app.use(express.json());
-app.use('/', eventRoute);
+app.use('/events', eventRoute);
const port = 3000;
```

แล้วลบ routing อื่นๆ ที่ไม่เกี่ยวข้องออกจาก server.ts ได้ครับ

Task 9 การสร้าง pagination

1. เปิด ไฟล์ EventRepositoryPrisma.ts เพื่อเพิ่ม function getAllEventsWithOrganizerpagination

```
+export function getAllEventsWithOrganizerPagination(  
+  pageSize: number,  
+  pageNo: number  
+) {  
+  return prisma.event.findMany({  
+    skip: pageSize * (pageNo - 1),  
+    take: pageSize,  
+    select: {  
+      id: true,  
+      category: true,  
+      title: true,  
+      organizerId: false,  
+      organizer: {  
+        select: {  
+          name: true  
+        }  
+      }  
+    }  
+  });  
+}
```

2. แก้ไขไฟล์ EventService.ts เพื่อเรียกใช้ function ที่สร้างขึ้นมาใหม่

```
+export function getAllEventsWithPagination(pageSize: number, pageNo: number)  
{  
+  return repo.getAllEventsWithOrganizerPagination(pageSize, pageNo);  
+}  
+
```

3. แก้ไข server.ts เพื่อให้รับค่า pageNo, pageSize เพื่อใช้ในการทำ pagination

```
router.get("/events", async(req, res) => {  
-  if (req.query.category) {  
+  if (req.query.pageSize && req.query.pageNo) {  
+    const pageSize = parseInt(req.query.pageSize as string);  
+    const pageNo = parseInt(req.query.pageNo as string);  
+    res.json(await service.getAllEventsWithPagination(pageSize, pageNo));  
+  } else if (req.query.category) {  
    const category = req.query.category;
```

4. ทดลองเรียก api ผ่าน apiDog โดยใช้ endpoint ดังนี้

GET

5. จากนั้นทดลองเปลี่ยน pageSize และ pageNo เพื่อสังเกตการเปลี่ยนแปลง

Task 10 การคืนค่า จำนวนทั้งหมดของ query

1. สร้าง function สำหรับการ นับจำนวนของข้อมูลทั้งหมดในฐานข้อมูล โดยเพิ่ม function ในไฟล์ EventRepositoryPrisma.ts

```
+export function countEvent() {  
+  return prisma.event.count();  
+}
```

2. เพิ่มการเรียก Count ใน EventService.ts

```
+export function count(){  
+  return repo.countEvent();  
+}
```

3. เพิ่มการคืนค่า count ใน EventRoute.ts

```
const pageNo = parseInt(req.query.pageNo as string);  
- res.json(await service.getAllEventsWithPagination(pageSize, pageNo));  
+ const events = await service.getAllEventsWithPagination(pageSize,  
pageNo);  
+ const totalEvents = await service.count();  
+ res.json({ totalEvents, events });  
+  
} else if (req.query.category) {
```

4. ทดลองเรียก api จะพบว่าข้อมูลที่ได้รับมีการเปลี่ยนแปลง

5. นอกจากวิธีนี้แล้วเรายังสามารถส่งข้อมูลจำนวนทั้งหมดผ่านทาง header ได้ โดยแก้ไข code ในส่วนของ eventRoute เป็นดังนี้

```
const events = await service.getAllEventsWithPagination(pageSize,  
pageNo);  
const totalEvents = await service.count();  
- res.json({ totalEvents, events });  
+ res.setHeader("x-total-count", totalEvents.toString());  
+ res.json(events);  
  
} else if (req.query.category) {  
const category = req.query.category;
```

6. ทดสอบส่งข้อมูล จากนั้นกดปุ่ม header ใน ApiDog เพื่อดูค่า header ที่ส่งออกมา



Body	Cookies	Headers 8	Console	Actual Request
Name	Value			
X-Powered-By	Express			
x-total-count	14			

Task 11 การสร้าง query จาก endpoint

1. ปกติแล้วเราไม่ควรคืนค่าทุกค่ากลับไปยังฐานข้อมูล ดังนั้นจึงควรมีค่า default pagination เพิ่มการสร้าง default pagination เมื่อผู้เรียกใช้ไม่ได้ระบุจำนวน pageSize และ pageNo มาโดยแก้ไขไฟล์ eventRoute.ts ดังนี้

```
router.get("/", async(req, res) => {  
  -   if (req.query.pageSize && req.query.pageNo) {  
  -     const pageSize = parseInt(req.query.pageSize as string);  
  -     const pageNo = parseInt(req.query.pageNo as string);  
  +  
  +   const pageSize = parseInt(req.query.pageSize as string) || 3;  
  +   const pageNo = parseInt(req.query.pageNo as string) || 1;  
    const events = await service.getAllEventsWithPagination(pageSize,  
pageNo);  
    const totalEvents = await service.count();
```

2. ปรับปรุง function เพื่อให้ query ข้อมูลผ่าน pagination ได้ ทั้งนี้เนื่องจากต้องมีการใช้ where ในการค้นหาทำให้รูปแบบเดิมที่เรียก count แยกต่างหาก จะทำให้ functions มีความซับซ้อน ดังนั้นจึงต้องแก้ไขให้ repository สามารถส่ง ค่าที่ต้องการบน pagination ได้ โดย เริ่มจากการสร้าง type ของผลลัพธ์ที่ต้องการก่อน โดย สร้างไฟล์ models/EventPage.ts โดยเพิ่ม type ดังนี้

```
import type {eventModel as Event} from "../generated/prisma/models/event";  
export interface PageEvent {  
  count: number;  
  events: Event[];  
}
```

3. แก้ไขไฟล์ eventRepositoryPrisma ให้สามารถรับค่า มาใช้ในการค้นหาทั้งการ search และการ count ดังนี้

```
-export function getAllEventsWithOrganizerPagination(  
+export async function getAllEventsWithOrganizerPagination(  
+ keyword: string,  
+ pageSize: number,  
+ pageNo: number  
) {  
+   const where = {  
+     title: { contains: keyword }  
+   }  
-   return prisma.event.findMany({  
+   const events = await prisma.event.findMany({  
+     where,  
+     skip: pageSize * (pageNo - 1),  
+     take: pageSize,  
+     select: {  
+       id: true,  
+       title: true,
```



```

        category: true,
        organizerId: false,
        organizer: {
          select: {
            name: true
          }
        }
      }
    }
  });
+ const count = await prisma.event.count({ where });
+ return { count, events } as unknown as PageEvent;

```

4. แก้ไข eventService.ts ให้เรียก functions ที่แก้ไข

```

-export function getAllEventsWithPagination(pageSize: number, pageNo: number)
{
- return repo.getAllEventsWithOrganizerPagination(pageSize, pageNo);
+export async function getAllEventsWithPagination(keyword: string, pageSize:
number, pageNo: number){
+ const pageEvents = await
repo.getAllEventsWithOrganizerPagination(keyword,pageSize, pageNo);
+ return pageEvents;

```

5. แก้ไข eventRoute เพื่อรองรับผลลัพธ์ที่เปลี่ยนแปลง และรับค่า keyword

```

const pageSize = parseInt(req.query.pageSize as string) || 3;
const pageNo = parseInt(req.query.pageNo as string) || 1;
- const events = await service.getAllEventsWithPagination(pageSize,
pageNo);
- const totalEvents = await service.count();
- res.setHeader("x-total-count", totalEvents.toString());
- res.json(events);
+ const keyword = req.query.keyword as string;
+ const result = await
service.getAllEventsWithPagination(keyword,pageSize, pageNo);
+
+ res.setHeader("x-total-count", result.count.toString());
+ res.json(result.events);
});

```

6. ทดลองยิง request โดยใช้ query

GET localhost:3000/events?pageSize=5&pageNo=1&keyword=con

ทดลองใช้ keyword ที่หลากหลายเพื่อดูผลลัพธ์ที่ได้

7. จะพบว่าการค้นหาเป็นแบบ case sensitive, หากต้องการเป็น case insensetive สามารถทำได้โดยการเพิ่ม mode ดังนี้

```

import {Prisma} from "../../generated/prisma/client";

const where = {
  title: {
    contains: keyword,
+ mode: 'insensitive'

```


Task 12 การสร้าง query ซับซ้อน

1. จากนั้นจะทดลองสร้าง query ที่สามารถใช้ keyword ในการค้นหาข้อความในแต่หลายๆ field โดยใช้ or operator แก้ไขไฟล์

eventRepositoryPrisma.ts

```
const where = {  
-   title: { contains: keyword }  
+   OR: [  
+       {title: {contains: keyword,  
+           mode: Prisma.QueryMode.insensitive}},  
+       {description: {contains: keyword}},  
+       {category: {contains: keyword}}  
+   ]  
}
```

จากนั้นทดลองค้นหาโดยใช้บางตัวอักษรที่มีในแต่ละส่วน

2. นอกจากนี้เราสามารถค้นหาข้อมูลจากเนื้อหาภายใน object หรือ nested query โดยการใช้ object ภายใน object, แก้ไขค่า ในส่วนของ where ดังนี้

```
OR: [  
  { title: { contains: keyword } },  
  { description: { contains: keyword } },  
-  { category: { contains: keyword } }  
+  { category: { contains: keyword } },  
+  { organizer: { name: { contains: keyword } } }  
]
```

จากนั้นทดลองใช้คำว่า CAMT เพื่อค้นหา พบอะไรบ้าง

Task Homework2 ทดลองสร้าง query ที่ซับซ้อน

1. จากระบบห้องสมุดที่พัฒนาเมื่อคาบที่แล้ว สร้าง query ที่เป็น pagination และค้นหาหนังสือโดยใช้ keyword ซึ่ง keyword จะค้นหาหนังสือได้จาก ชื่อหนังสือ หมวดหนังสือ ชื่อผู้แต่ง และชื่อของผู้เคยยืมหนังสือ โดยให้เลือกแสดงข้อมูลตามความเหมาะสม

Task 13 การใช้ Response Code

1. การใช้ response code เพื่อแสดงสถานะที่ไม่มีข้อมูลที่ชัดเจน เช่น ถ้า page เกินกว่าที่กำหนดเช่นใน query นี้

GET localhost:3000/events?pageSize=1&pageNo=5&keyword=CAMT

ระบบจะคืน list ว่างเนื่องจาก pagination cursor ไปเกินกว่าความเป็นจริง แต่เราไม่สามารถสื่อสารปัญหาไปยัง front end ได้

2. แก้ไข eventRoutes เพื่อให้ใช้ response code เพื่อแจ้งให้ frontend รับทราบว่าเปิดปัญหาอะไรขึ้น

```
const result = await
```

```
service.getAllEventsWithPagination(keyword, pageSize, pageNo);
```

```
-  
+   if (result.events.length === 0) {  
+       res.status(404).send("No event found");  
+       return;  
+   }  
+   res.setHeader("x-total-count", result.count.toString());
```

จากนั้นทดลอง query อีกครั้งสังเกต response code ที่ได้ทางด้านขวาล่าง

Task 14 การใช้ Error Handling - Try Catch

1. ทดลองค้นหาข้อมูลโดยใช้ query นี้

GET localhost:3000/events?pageSize=1&pageNo=-1&keyword=CAMT Send

get all events with pagination with keyword

Params 3 Body Headers 7 Cookies Auth Pre Processors Post Processors Settings

Query Params

Name	Value
✓ pageSize	= 1
✓ pageNo	= -1
✓ keyword	= CAMT

จะพบว่า server หยุดการทำงาน เราต้อง restart การทำงานด้วยสั่ง `npm run dev` อีกครั้ง

2. เพื่อป้องกันไม่ให้ server หยุดการทำงานเนื่องจากข้อผิดพลาดต่างๆ ที่อาจเกิดขึ้นได้จึงใช้ try catch เพื่อดักจับปัญหาที่อาจเกิดขึ้นได้ แก้ไข code ส่วน eventRoute เพื่อดักจับปัญหาและคืนผลลัพธ์ของปัญหา

```
const keyword = req.query.keyword as string;
-   const result = await
service.getAllEventsWithPagination(keyword, pageSize, pageNo);
-   if (result.events.length === 0) {
-       res.status(404).send("No event found");
-       res.setHeader("x-total-count", result.count.toString());
-       res.json(result.events);
+   try{
+       const result = await
service.getAllEventsWithPagination(keyword, pageSize, pageNo);
+       if (result.events.length === 0) {
+           res.status(404).send("No event found");
+           return;
+       }
+       res.setHeader("x-total-count", result.count.toString());
+       res.json(result.events);
+   } catch (error) {
+       res.status(500).send("Internal Server Error");
+       return;
+   }
+
+ });
```

ทดลองส่งข้อมูลที่มีปัญหาอีกครั้งเพื่อดูผลลัพธ์

3. ทั้งนี้ สถานะ 500 ไม่ได้บอกชัดเจนว่ามีปัญหาอะไร หากเราทราบปัญหาสามารถแจ้งปัญหาไปยังผู้ใช้เพื่อแก้ไขปัญหาให้เหมาะสมได้

```
-   } catch (error) {
-       res.status(500).send("Internal Server Error");
-       return;
+ } catch (error) {
+   if (pageNo < 1 || pageSize < 1) {
+       res.status(400).send("Invalid pageNo or pageSize");
+   } else {
```

```
+     res.status(500).send("Internal Server Error");  
+   }  
+   return;
```

ทดลองยิง api อีกครั้งดูผลลัพธ์ที่ได้

Task 15 การใช้งาน Error Handling - finally

8. Finally ใช้ในการแจ้งสถานะที่จะมีการเรียกทุกครั้ง ทดลองแก้ไข code ที่ eventRoute เพื่อเพิ่ม finally

```
        return;  
+    } finally {  
+        console.log(`Request is completed. with pageNo=${pageNo} and  
pageSize=${pageSize}`);  
    }
```

ทดลองเรียก api ทั้งในแบบปกติ และแบบที่เกิดปัญหาลองดูผลลัพธ์ที่ได้
ผ่านทาง terminal

ทดลองเพิ่ม try catch ใน endpoint ของห้องสมุด และทดสอบการเรียก
api ทั้งในส่วนที่ดี และไม่ดี

Task 10 การสร้าง endpoint เพื่อ query ข้อมูล

1. สามารถใช้ query parameter เพื่อช่วยกรองข้อมูล (filter) ข้อมูลที่ต้องการได้ แก้ไข /events endpoint ด้วย code ชุดนี้
- 2.

